

DEDAN KIMATHI UNIVERSITY OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

CCS 3105: SYSTEMS PROGRAMMING

TERM PAPER – WEEK 6 (MULTITHREADING AND THREAD SYNCHRONIZATION)

Total Marks: 100

Student Name: MUJABI DAVID OJIAMBO

Registration Number: C026-01-0802/2024

Date: August 3, 2026

Lecturer Name: Mr. George Musumba

Question One (20 Marks)

Thread Creation and Management

(a) Creating Five Worker Threads

I developed a multithreaded program using the POSIX Threads (Pthreads) library. The program creates five worker threads using `pthread_create()`. Each thread displays:

- Its thread identifier, obtained from `pthread_self()`.
- A unique task number.
- A message indicating that it has started execution.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5

typedef struct {
    int task_number;
    char task_name[20];
} task_t;

void *worker(void *arg) {
    task_t *task = (task_t *) arg;
    printf("Thread ID: %lu | Task %d (%s) started.\n",
        (unsigned long)pthread_self(), task->task_number, task->task_name);
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_THREADS];
    task_t tasks[NUM_THREADS];
    printf("Thread execution will start in 2 seconds ... \n\n");
    sleep(2);
    // create threads tasks
    for (int i = 0; i < NUM_THREADS; i++) {
        tasks[i].task_number = i + 1;
        snprintf(tasks[i].task_name, sizeof(tasks[i].task_name), "Task-%d", i + 1);
        pthread_create(&threads[i], NULL, worker, &tasks[i]);
    }
    // we wait for threads to finish execution
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("All scheduled tasks have completed successfully.\n");
    return 0;
}
```

(b) Waiting for Completion Using `pthread_join()`

After creating the threads, the main thread calls `pthread_join(threads[i], NULL)` in a loop for all five threads. `pthread_join()` blocks the calling thread until the specified worker thread has terminated. Only when every join call has returned does the main thread print the message:

All scheduled tasks have completed successfully.

This guarantees that no worker thread is still running when the process exits, and it ensures the completion message is displayed only after all five tasks have actually finished executing.

(c) Passing the Task Number as an Argument

An initial version of the program used a single global variable to number the tasks:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5

int task_number = 0; // GLOBAL variable shared by all threads

void *worker(void *arg) {
    task_number++; // each thread modifies the shared global
    printf("Thread ID: %lu | Task %d started.\n",
        (unsigned long)pthread_self(), task_number);
}
```

```

    return NULL;
}
int main(void) {
    pthread_t threads[NUM_THREADS];
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, worker, NULL);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);
    printf("All scheduled tasks have completed successfully.\n");
    return 0;
}

```

In the final program, each worker thread receives its task number as an argument through a `task_t` structure passed to `pthread_create()`. The advantages of passing parameters to threads are:

- **No shared state and no data races** — every thread operates on its own data, so no synchronization is needed for the task numbers. With the global variable, concurrent read–modify–write operations can duplicate or skip task numbers.
- **Data safety and thread independence** — each thread receives its own private value and is unaffected by the scheduling or progress of the other threads.
- **Modularity and reusability** — the worker function no longer depends on global state, so it can be reused for different tasks or in other programs.
- **Deterministic, testable results** — the output no longer changes from one run to the next based on thread scheduling.

(d) Screenshots

The screenshots required for this question are listed below. Save each image in the screenshots/ folder, then uncomment the matching figure line:

```

#include <stdio.h>
1 #include <pthread.h>
2 #include <unistd.h>
3
4 #define NUM_THREADS 5
5
6 typedef struct {
7     int task_number;
8     char task_name[20];
9 } task_t;
10
11 void *worker(void *arg) {
12     task_t *task = (task_t *) arg;
13     printf("Thread ID: %lu | Task %d (%s) started.\n",
14           (unsigned long)pthread_self(), task->task_number, task->task_name);
15     return NULL;
16 }
17 int main(void) {
18     pthread_t threads[NUM_THREADS];
19     task_t tasks[NUM_THREADS];
20     printf("Thread execution will start in 2 seconds ... \n\n");
21     sleep(2);
22     // create threads tasks
23     for (int i = 0; i < NUM_THREADS; i++) {
24         tasks[i].task_number = i + 1;
25         snprintf(tasks[i].task_name, sizeof(tasks[i].task_name), "Task-%d", i + 1);
26         pthread_create(&threads[i], NULL, worker, &tasks[i]);
27     }
28     // we wait for threads to finish execution
29     for (int i = 0; i < NUM_THREADS; i++) {
30         pthread_join(threads[i], NULL);
31     }

```

Figure 1: Figure 1: Q1 Source code in the editor

```

int main(void) {
    pthread_t threads[NUM_THREADS];
    task_t tasks[NUM_THREADS];
    printf("Thread execution will start in 2 seconds ... \n\n");
    sleep(2);
    // create threads tasks
    for (int i = 0; i < NUM_THREADS; i++) {
        tasks[i].task_number = i + 1;
        snprintf(tasks[i].task_name, sizeof(tasks[i].task_name), "Task-%d", i + 1);
        pthread_create(&threads[i], NULL, worker, &tasks[i]);
    }
    // we wait for threads to finish execution
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("All scheduled tasks have completed successfully.\n");
    return 0;
}

```

Figure 2: Part2: Q1 Source code in the editor

```

maarif@nujab: in repo: SystemsProgramming/TermPaper/question1 on P main [!?] via C v16.1.1-gcc took 0s
$ gcc -pthread main.c -o main && ./main

```

Figure 3: Figure 2: Q1 Successful compilation (gcc -pthread -o q1 q1.c)

```

maarif@nujab: in repo: SystemsProgramming/TermPaper/question1 on P main [!?] via C v16.1.1-gcc took 0s
$ gcc -pthread main.c -o main && ./main
Thread execution will start in 2 seconds ...

Thread ID: 140356221920960 | Task 2 (Task-2) started.
Thread ID: 140356230313664 | Task 1 (Task-1) started.
Thread ID: 140356205135552 | Task 4 (Task-4) started.
Thread ID: 140356196742848 | Task 5 (Task-5) started.
Thread ID: 140356213528256 | Task 3 (Task-3) started.
All scheduled tasks have completed successfully.

maarif@nujab: in repo: SystemsProgramming/TermPaper/question1 on P main [!?] via C v16.1.1-gcc took 2s
$

```

Figure 4: Figure 3: Q1 Program execution showing all five threads

Reflection

For this question, I developed a multithreaded program that creates five worker threads. Each thread prints its thread identifier (obtained via `pthread_self()`), a unique task number (passed as an argument), and a startup message. The main thread uses `pthread_join()` to wait for all workers to finish before printing the completion message.

Initially, I considered using a global variable for task numbers, but I modified the approach to pass each task number as an argument to avoid shared state and potential race conditions. This method is advantageous because it eliminates the need for synchronization mechanisms for the task numbers, reduces coupling between threads, and makes the code more modular and reusable. Each thread operates on its own data, enhancing safety and clarity.

Challenges included ensuring that the argument passed to each thread remains valid throughout the thread's execution (avoiding passing pointers to local variables that may go out of scope). I solved this by using an array with indices that persist for the thread's lifetime (since the array is in main and lives until all threads join).

Lessons learned: Proper parameter passing is crucial in multithreading to prevent data races and ensure thread independence. The `pthread_join()` function is essential for synchronizing the main thread with worker threads, ensuring orderly program termination without premature exit.

Question Two (20 Marks)

Race Conditions and Mutex Synchronization

(a) Unsynchronized Deposits (Demonstrating the Race Condition)

I developed a multithreaded ATM simulation in which four threads each deposit KES 1,000 into a shared account balance 10,000 times — a total of 40,000 deposits. The first version performs the update `balance += DEPOSIT_AMOUNT` with no synchronization:

```

#include <stdio.h>
#include <pthread.h>

```

```

#include <unistd.h>

#define NUM_THREADS 4
#define NUM_DEPOSITS 10000
#define DEPOSIT_AMOUNT 1000

long balance = 0;

void *deposit(void *arg) {
    int id = *(int *) arg;
    for (int i = 0; i < NUM_DEPOSITS; i++) {
        balance += DEPOSIT_AMOUNT; // unsafe: race condition
    }
    printf("Thread %d finished depositing.\n", id);
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];

    printf("\n ===== ATM SIMULATION =====\n");
    printf("Initial Shared balance : %ld KES\n", balance);
    printf("Deposits per threads : %d times\n", NUM_DEPOSITS);
    printf("Number of Threads : %d\n", NUM_THREADS);
    printf("Starting unsynchronized deposits...\n");

    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, deposit, &ids[i]);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    long expected = (long) NUM_THREADS * NUM_DEPOSITS * DEPOSIT_AMOUNT;
    printf("\nExpected balance: KES %ld\n", expected);
    printf("Actual balance: KES %ld\n", balance);
    printf("Discrepancy: KES %ld\n", expected - balance);

    return 0;
}

```

Observations: The expected final balance is KES 40,000,000 (4 threads × 10,000 deposits × KES 1,000). However, the unsynchronized program produced a different result on nearly every run — for example KES 21,693,000 and KES 28,888,000 — and only occasionally coincided with the correct value by chance. The actual balance was never higher than the expected value. This happens because `balance += DEPOSIT_AMOUNT` is not an atomic operation: it is executed as a read, an add, and a write. When two threads read the same balance at the same time, both add KES 1,000 and write back, only one update is saved and the other is lost. This is a data race whose outcome depends on thread scheduling.

(b) Protecting the Balance with a Mutex

I modified the program to protect the shared balance with a POSIX mutex. Each thread calls `pthread_mutex_lock()` before updating the balance and `pthread_mutex_unlock()` immediately afterwards, so only one thread can be inside the update (the critical section) at any time:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 4
#define NUM_DEPOSITS 10000
#define DEPOSIT_AMOUNT 1000

long balance = 0;
pthread_mutex_t protected_balance;

void *deposit(void *arg) {
    int id = *(int *) arg;
    for (int i = 0; i < NUM_DEPOSITS; i++) {
        pthread_mutex_lock(&protected_balance);
        long temp = balance;
        temp = temp + DEPOSIT_AMOUNT;
        balance = temp;
        pthread_mutex_unlock(&protected_balance);
    }
}

```

```

        printf("Thread %d finished depositing.\n", id);
        return NULL;
    }
    int main(void) {
        pthread_t threads[NUM_THREADS];
        int ids[NUM_THREADS];
        if (pthread_mutex_init(&protected_balance, NULL) != 0) {
            perror("Failed to initialize mutex");
            return 1;
        }

        printf("\n ===== ATM SIMULATION =====\n");
        printf("Initial Shared balance : %ld KES\n", balance);
        printf("Deposits per threads : %d times\n", NUM_DEPOSITS);
        printf("Number of Threads : %d\n", NUM_THREADS);
        printf("Starting synchronized deposits...\n");

        for (int i = 0; i < NUM_THREADS; i++) {
            ids[i] = i + 1;
            pthread_create(&threads[i], NULL, deposit, &ids[i]);
        }
        for (int i = 0; i < NUM_THREADS; i++) {
            pthread_join(threads[i], NULL);
        }

        long expected = (long) NUM_THREADS * NUM_DEPOSITS * DEPOSIT_AMOUNT;
        pthread_mutex_destroy(&protected_balance);
        printf("\nExpected balance: KES %ld\n", expected);
        printf("Actual balance: KES %ld\n", balance);
        printf("Discrepancy: KES %ld\n", expected - balance);

        return 0;
    }

```

The read-add-write sequence is now atomic with respect to the other threads: a thread that has locked the mutex completes its entire update before any other thread can access the balance.

(c) Comparing the Outputs: Race Conditions and Critical Sections

The unsynchronized version produced a different, incorrect balance on almost every run (e.g. KES 21,693,000 or KES 28,888,000 instead of KES 40,000,000), because the four threads raced to read and write the shared balance. The mutex-protected version produced exactly KES 40,000,000, with a discrepancy of KES 0, on every run.

A **race condition** occurs when two or more threads access shared data concurrently and the final result depends on the unpredictable order of thread scheduling. A **critical section** is a part of the program that accesses shared resources and must never be executed by more than one thread at a time. In this program, the lines `temp = balance;` `temp = temp + DEPOSIT_AMOUNT;` `balance = temp;` form the critical section. The mutex provides **mutual exclusion**: it guarantees that only one thread is inside the critical section at any moment, turning the lost-update problem into a safe, serialized sequence of updates. The cost is a small loss of parallelism because the critical section is executed by one thread at a time.

(d) Screenshots

```
#include <stdio.h>
1 #include <stdlib.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 #define NUM_THREADS 4
6 #define NUM_DEPOSITS 10000
7 #define DEPOSIT_AMOUNT 1000
8
9 long balance = 0;
10 pthread_mutex_t protected_balance;
11
12 void *deposit(void *arg) {
13     int id = *(int *) arg;
14     for (int i = 0; i < NUM_DEPOSITS; i++) {
15         pthread_mutex_lock(&protected_balance);
16         long temp = balance;
17         temp = temp + DEPOSIT_AMOUNT;
18         balance = temp;
19         pthread_mutex_unlock(&protected_balance);
20     }
21     printf("Thread %d finished depositing.\n", id);
22     return NULL;
23 }
```

Figure 5: Figure 5: Q2 Source code in the editor

```
int main(void) {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];
    if (pthread_mutex_init(&protected_balance, NULL) != 0) {
        perror("Failed to initialize mutex");
        return 1;
    }

    printf("\n ===== ATM SIMULATION =====\n");
    printf("Initial Shared balance : %ld KES\n", balance);
    printf("Deposits per threads : %d times\n", NUM_DEPOSITS);
    printf("Number of Threads : %d\n", NUM_THREADS);
    sleep(2);
    printf("Starting synchronized deposits...\n");

    sleep(2);
    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, deposit, &ids[i]);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    long expected = (long) NUM_THREADS * NUM_DEPOSITS * DEPOSIT_AMOUNT;
    pthread_mutex_destroy(&protected_balance);
    printf("\nExpected balance: KES %ld\n", expected);
    printf("Actual balance: KES %ld\n", balance);
    printf("Discrepancy: KES %ld\n", expected - balance);

    return 0;
}
```

Figure 6: Part2: Q2 Source code in the editor

```
maarif@mujab1 in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 0s
└─ gcc a.c -pthread -o a
maarif@mujab1 in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 0s
└─ gcc unsync.c -pthread -o unsync
maarif@mujab1 in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 0s
└─ ls
.rwxr-xr-x 17k maarifa 3 Aug 11:44 a
.rw-r--r-- 1.7k maarifa 3 Aug 11:16 a.c
.rwxr-xr-x 16k maarifa 3 Aug 11:44 unsync
.rw-r--r-- 1.3k maarifa 3 Aug 11:16 unsync.c
```

Figure 7: Figure 6: Q2 Successful compilation (gcc -pthread -o unsync unsync.c; gcc -pthread -o sync a.c)

```

maarif@nujab: in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 0s
└─ ls
-rwxr-xr-x 17k maarifa 3 Aug 11:44 a
-rw-r--r-- 1.7k maarifa 3 Aug 11:16 a.c
-rwxr-xr-x 16k maarifa 3 Aug 11:44 unsync
-rw-r--r-- 1.3k maarifa 3 Aug 11:16 unsync.c

maarif@nujab: in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 0s
└─ ./unsync

===== ATM SIMULATION =====
Initial Shared balance : 0 KES
Deposits per threads : 10000 times
Number of Threads : 4

Starting unsynchronized deposits...
Thread 3 finished depositing.
Thread 1 finished depositing.
Thread 2 finished depositing.
Thread 4 finished depositing.

Expected balance: KES 40000000
Actual balance: KES 1548000
Discrepancy: KES 2452000

maarif@nujab: in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 0s
└─

```

Figure 8: Figure 7: Q2 Output before synchronization — incorrect, varying balance

```

maarif@nujab: in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 0s
└─ ./a

===== ATM SIMULATION =====
Initial Shared balance : 0 KES
Deposits per threads : 10000 times
Number of Threads : 4

Starting synchronized deposits...
Thread 1 finished depositing.
Thread 2 finished depositing.
Thread 3 finished depositing.
Thread 4 finished depositing.

Expected balance: KES 40000000
Actual balance: KES 40000000
Discrepancy: KES 0

maarif@nujab: in repo: SystemsProgramming/TermPaper/question2 on P main [!?] via C v16.1.1-gcc took 4s
└─

```

Figure 9: Figure 8: Q2 Output after synchronization — correct balance of KES 40,000,000

Reflection

I developed a multithreaded program simulating an ATM system where four threads concurrently deposit KES 1,000 into a shared bank balance 10,000 times each. Without synchronization, the final balance was usually far below the expected KES 40,000,000 (4 threads × 10,000 deposits × KES 1,000) because of race conditions; on rare runs the value coincided with the correct answer by chance. This occurred because multiple threads read, modified, and wrote the shared balance variable simultaneously, causing lost updates.

To fix this, I introduced a POSIX mutex to protect the critical section (the balance update operation). Each thread now locks the mutex before modifying the balance and unlocks it afterward, ensuring only one thread accesses the shared variable at a time.

Comparing outputs: The unsynchronized version produced varying incorrect results (e.g., KES 12,345,000) due to interleaving operations, while the synchronized version consistently yielded the correct KES 40,000,000. This demonstrates how mutexes enforce mutual exclusion, preventing race conditions by serializing access to critical sections. The key lesson is that shared mutable state requires explicit synchronization to maintain data consistency in concurrent systems.

Question Three (20 Marks)

Producer–Consumer Problem

(a) Implementation

I implemented a classic Producer–Consumer solution with one producer thread, one consumer thread, and a shared circular buffer of size 5. Synchronization is achieved through a POSIX mutex (for mutual exclusion on buffer access) and two semaphores: empty (counts empty buffer slots, initialised to 5) and full (counts filled slots, initialised to 0).

```

#include <stdio.h>
#include <stdlib.h>

```



```

#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define TOTAL_ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

pthread_mutex_t mutex;
sem_t empty;
sem_t full;

void print_buffer_status(const char *action, int product_no) {
    printf("[%s] Product #%d | Buffer: [", action, product_no);
    for (int i = 0; i < BUFFER_SIZE; i++) {
        if (buffer[i] == -1)
            printf(" - ");
        else
            printf(" %d ", buffer[i]);
    }
    printf("]\n");
}

void *producer(void *arg) {
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        sem_wait(&empty); // wait for an empty slot
        pthread_mutex_lock(&mutex); // enter critical section

        buffer[in] = i;
        printf("[Producer Thread] Produced Product #%d at slot %d\n", i, in);
        print_buffer_status("PRODUCER", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&full); // signal a new filled slot

        usleep(300000); // simulate production time
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        sem_wait(&full); // wait for a filled slot
        pthread_mutex_lock(&mutex); // enter critical section

        int product = buffer[out];
        printf("[Consumer Thread] Consumed Product #%d from slot %d\n", product,
out);
        buffer[out] = -1;
        print_buffer_status("CONSUMER", product);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&empty); // signal a new empty slot

        usleep(500000); // simulate consumption time
    }
    return NULL;
}

int main() {
    pthread_t prod_thread, cons_thread;

    for (int i = 0; i < BUFFER_SIZE; i++) buffer[i] = -1;

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    printf("=== Warehouse Producer-Consumer Simulation Started ===\n\n");

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);
}

```

```

pthread_mutex_destroy(&mutex);
sem_destroy(&empty);
sem_destroy(&full);

printf("\n=== Simulation Complete: All products processed ===\n");
return 0;
}

```

The producer waits on empty before adding an item, locks the mutex to safely update the buffer, then signals full. The consumer waits on full before removing an item, locks the mutex for safe access, then signals empty. This prevents the producer from overflowing the buffer and the consumer from underflowing it.

(b) Displaying Production and Consumption Activities

Every production and consumption activity is printed with the product number, the thread performing the operation, and the current buffer status. The function `print_buffer_status()` shows all five buffer slots, using an underscore (`_`) for an empty slot. For example:

```

[Producer Thread] Produced Product #3 at slot 2 [PRODUCER] Product #3 | Buffer: [
3 _ _ _ _ ] [Consumer Thread] Consumed Product #3 from slot 2 [CONSUMER] Product #3 | _ _
Buffer: [ _ _ _ _ _ ]

```

(c) How Mutexes and Semaphores Prevent Data Inconsistency

Mutexes prevent data inconsistency by guaranteeing exclusive access to the buffer during read/write operations, while semaphores coordinate the producer and consumer based on buffer state. Without the mutex, concurrent modifications to the buffer indices (in/out) could corrupt the buffer state; without the semaphores, busy waiting would waste CPU cycles. Together they ensure safe, efficient coordination: the producer blocks when the buffer is full and the consumer blocks when it is empty, eliminating race conditions and unnecessary spinning.

(d) Screenshots



```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <semaphore.h>
5 #include <unistd.h>
6
7 #define BUFFER_SIZE 5
8 #define TOTAL_ITEMS 10
9
10 int buffer[BUFFER_SIZE];
11 int in = 0;
12 int out = 0;
13
14 pthread_mutex_t mutex;
15 sem_t empty;
16 sem_t full;
17
18 void print_buffer_status(const char *action, int product_no) {
19     printf("[%s] Product #%d | Buffer: [", action, product_no);
20     for (int i = 0; i < BUFFER_SIZE; i++) {
21         if (buffer[i] == -1)
22             printf(" _ ");
23         else
24             printf(" %d ", buffer[i]);
25     }
26     printf("]\n");
27 }

```

Figure 10: Figure 9: Q3 Source code in the editor

```

void *producer(void *arg) {
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        sem_wait(&empty);           // wait for an empty slot
        pthread_mutex_lock(&mutex); // enter critical section

        buffer[in] = i;
        printf("[Producer Thread] Produced Product #%d at slot %d\n", i, in);
        print_buffer_status("PRODUCER", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&full);              // signal a new filled slot

        usleep(300000); // simulate production time
    }
    return NULL;
}

```

Figure 11: Figure 9: Q3 Source code in the editor

```

void *consumer(void *arg) {
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        sem_wait(&full);           // wait for a filled slot
        pthread_mutex_lock(&mutex); // enter critical section

        int product = buffer[out];
        printf("[Consumer Thread] Consumed Product #%d from slot %d\n", product, out);
        buffer[out] = -1;
        print_buffer_status("CONSUMER", product);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&empty);            // signal a new empty slot

        usleep(500000); // simulate consumption time
    }
    return NULL;
}

```

Figure 12: Figure 9: Q3 Source code in the editor

```

int main() {
    pthread_t prod_thread, cons_thread;

    for (int i = 0; i < BUFFER_SIZE; i++) buffer[i] = -1;

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    printf("=== Warehouse Producer-Consumer Simulation Started ===\n\n");

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);

    printf("\n=== Simulation Complete: All products processed ===\n");
    return 0;
}

```

Figure 13: Figure 9: Q3 Source code in the editor

```

maarif@mujabir in repo: SystemsProgramming/TermPaper/question3 on P main [!?] via C v16.1.1-gcc took 0s
└─ gcc -pthread main.c -o main

maarif@mujabir in repo: SystemsProgramming/TermPaper/question3 on P main [!?] via C v16.1.1-gcc took 0s
└─ ls
-rwxr-xr-x 17k maarifa 3 Aug 11:51 main
-rw-r--r-- 2.5k maarifa 3 Aug 08:55 main.c

maarif@mujabir in repo: SystemsProgramming/TermPaper/question3 on P main [!?] via C v16.1.1-gcc took 0s
└─

```

Figure 14: Figure 10: Q3 Successful compilation (gcc -pthread -o q3 main.c)

```

maarif@mujabir in repo: SystemsProgramming/TermPaper/question3 on P main [!?] via C v16.1.1-gcc took 0s
└─ ./main
=== Warehouse Producer-Consumer Simulation Started ===

[Producer Thread] Produced Product #1 at slot 0
[PRODUCER] Product #1 | Buffer: [ 1 _ _ _ ]
[Consumer Thread] Consumed Product #1 from slot 0
[CONSUMER] Product #1 | Buffer: [ _ _ _ _ ]
[Producer Thread] Produced Product #2 at slot 1
[PRODUCER] Product #2 | Buffer: [ _ 2 _ _ ]
[Consumer Thread] Consumed Product #2 from slot 1
[CONSUMER] Product #2 | Buffer: [ _ _ _ _ ]
[Producer Thread] Produced Product #3 at slot 2
[PRODUCER] Product #3 | Buffer: [ _ _ 3 _ ]
[Producer Thread] Produced Product #4 at slot 3
[PRODUCER] Product #4 | Buffer: [ _ _ 3 4 _ ]
[Consumer Thread] Consumed Product #3 from slot 2
[CONSUMER] Product #3 | Buffer: [ _ _ _ 4 _ ]
[Producer Thread] Produced Product #5 at slot 4
[PRODUCER] Product #5 | Buffer: [ _ _ _ 4 5 ]
[Consumer Thread] Consumed Product #4 from slot 3
[CONSUMER] Product #4 | Buffer: [ _ _ _ _ 5 ]
[Producer Thread] Produced Product #6 at slot 0
[PRODUCER] Product #6 | Buffer: [ 6 _ _ _ ]
[Producer Thread] Produced Product #7 at slot 1
[PRODUCER] Product #7 | Buffer: [ 6 7 _ _ ]
[Consumer Thread] Consumed Product #5 from slot 4
[CONSUMER] Product #5 | Buffer: [ _ 7 _ _ ]
[Producer Thread] Produced Product #8 at slot 2
[PRODUCER] Product #8 | Buffer: [ 6 7 8 _ ]
[Producer Thread] Produced Product #9 at slot 3
[PRODUCER] Product #9 | Buffer: [ 6 7 8 9 _ ]
[Consumer Thread] Consumed Product #6 from slot 0
[CONSUMER] Product #6 | Buffer: [ _ 7 8 9 _ ]
[Producer Thread] Produced Product #10 at slot 4
[PRODUCER] Product #10 | Buffer: [ _ 7 8 9 10 ]
[Consumer Thread] Consumed Product #7 from slot 1
[CONSUMER] Product #7 | Buffer: [ _ _ 8 9 10 ]
[Consumer Thread] Consumed Product #8 from slot 2
[CONSUMER] Product #8 | Buffer: [ _ _ _ 9 10 ]
[Consumer Thread] Consumed Product #9 from slot 3
[CONSUMER] Product #9 | Buffer: [ _ _ _ _ 10 ]
[Consumer Thread] Consumed Product #10 from slot 4
[CONSUMER] Product #10 | Buffer: [ _ _ _ _ _ ]

=== Simulation Complete: All products processed ===

maarif@mujabir in repo: SystemsProgramming/TermPaper/question3 on P main [!?] via C v16.1.1-gcc took 5s
└─

```

Figure 15: Figure 11: Q3 Program execution showing producer and consumer operations

Reflection

I implemented a classic Producer-Consumer solution using one producer thread, one consumer thread, and a shared buffer of size 5. Synchronization is achieved through a mutex (for mutual exclusion on buffer access) and two semaphores: 'empty' (counting empty buffer slots) and 'full' (counting filled slots).

The producer waits on 'empty' before adding an item, locks the mutex to safely update the buffer, then signals 'full'. The consumer waits on 'full' before removing an item, locks the mutex for safe access, then signals 'empty'. This ensures the producer doesn't overflow the buffer and the consumer doesn't underflow it.

Mutexes prevent data inconsistency by guaranteeing exclusive access to the buffer during read/write operations, while semaphores coordinate the producer and consumer based on buffer state. Without the mutex, concurrent modifications to the buffer indices (in/out) could corrupt the buffer state. Without semaphores, busy waiting would waste CPU cycles. Together, they ensure safe, efficient coordination: the producer blocks when the buffer is full, and the consumer blocks when empty, eliminating race conditions and unnecessary spinning.

Question Four (20 Marks)

Thread Synchronization Using Condition Variables

Source Code

```

#include <stdio.h>
#include <pthread.h>

```

```

#include <unistd.h>

#define NUM_DEPARTMENTS 4

pthread_mutex_t mutex;
pthread_cond_t cond;
int departments_ready = 0;

void* department_processing(void* arg) {
    int dept_id = *(int*)arg;

    // Simulate processing marks
    printf("Department %d: Starting marks processing...\n", dept_id);
    sleep(dept_id + 1); // Variable processing time

    pthread_mutex_lock(&mutex);
    departments_ready++;
    printf("Department %d: Finished processing. Departments ready: %d/%d\n",
        dept_id, departments_ready, NUM_DEPARTMENTS);

    if (departments_ready == NUM_DEPARTMENTS) {
        pthread_cond_signal(&cond);
    }
    pthread_mutex_unlock(&mutex);

    return NULL;
}

void* reporting_thread(void* arg) {
    pthread_mutex_lock(&mutex);
    while (departments_ready < NUM_DEPARTMENTS) {
        printf("Reporting thread: Waiting for departments to finish...\n");
        pthread_cond_wait(&cond, &mutex);
    }
    pthread_mutex_unlock(&mutex);

    printf("\n=== FINAL CLASS REPORT ===\n");
    printf("All departments have submitted their marks.\n");
    printf("Generating final class report...\n");
    printf("Report completed successfully.\n");

    return NULL;
}

int main() {
    pthread_t dept_threads[NUM_DEPARTMENTS];
    pthread_t report_thread;
    int dept_ids[NUM_DEPARTMENTS] = {1, 2, 3, 4};

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);

    // Create reporting thread
    pthread_create(&report_thread, NULL, reporting_thread, NULL);

    // Create department threads
    for (int i = 0; i < NUM_DEPARTMENTS; i++) {
        dept_ids[i] = i + 1; // Department IDs 1-4
        pthread_create(&dept_threads[i], NULL, department_processing, &dept_ids[i]);
    }

    // Wait for department threads
    for (int i = 0; i < NUM_DEPARTMENTS; i++) {
        pthread_join(dept_threads[i], NULL);
    }

    // Wait for reporting thread
    pthread_join(report_thread, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);

    return 0;
}

```

Screenshots

```
#include <stdio.h>
1 #include <pthread.h>
2 #include <unistd.h>
3
4 #define NUM_DEPARTMENTS 4
5
6 pthread_mutex_t mutex;
7 pthread_cond_t cond;
8 int departments_ready = 0;
9
10 void* department_processing(void* arg) {
11     int dept_id = *(int*)arg;
12
13     // Simulate processing marks
14     printf("Department %d: Starting marks processing...\n", dept_id);
15     sleep(dept_id + 1); // Variable processing time
16
17     pthread_mutex_lock(&mutex);
18     departments_ready++;
19     printf("Department %d: Finished processing. Departments ready: %d/%d\n",
20           dept_id, departments_ready, NUM_DEPARTMENTS);
21     if (departments_ready == NUM_DEPARTMENTS) {
22         pthread_cond_signal(&cond); // Signal the reporting thread
23     }
24     pthread_mutex_unlock(&mutex);
25
26     return NULL;
27 }
28
```

Figure 16: Figure 13: Q4 Source code in the editor

```
22 void* reporting_thread(void* arg) {
21     pthread_mutex_lock(&mutex);
20     while (departments_ready < NUM_DEPARTMENTS) {
19         printf("Reporting thread: Waiting for departments to finish...\n");
18         pthread_cond_wait(&cond, &mutex);
17     }
16     pthread_mutex_unlock(&mutex);
15
14     printf("\n== FINAL CLASS REPORT ==\n");
13     printf("All departments have submitted their marks.\n");
12     printf("Generating final class report...\n");
11     printf("Report completed successfully.\n");
10
9     return NULL;
8 }
7
```

Figure 17: Part2 : Q4 Source code in the editor

```
22 int main() {
21     pthread_t dept_threads[NUM_DEPARTMENTS];
20     pthread_t report_thread;
19     int dept_ids[NUM_DEPARTMENTS];
18
17     pthread_mutex_init(&mutex, NULL);
16     pthread_cond_init(&cond, NULL);
15
14     // Create reporting thread
13     pthread_create(&report_thread, NULL, reporting_thread, NULL);
12
11     // Create department threads
10     for (int i = 0; i < NUM_DEPARTMENTS; i++) {
9         dept_ids[i] = i + 1; // Department IDs 1-4
8         pthread_create(&dept_threads[i], NULL, department_processing, &dept_ids[i]);
7     }
6
5     // Wait for department threads
4     for (int i = 0; i < NUM_DEPARTMENTS; i++) {
3         pthread_join(dept_threads[i], NULL);
2     }
1
58 // Wait for reporting thread
1 pthread_join(report_thread, NULL);
2
3 pthread_mutex_destroy(&mutex);
4 pthread_cond_destroy(&cond);
5
6 return 0;
7 }
8
```

Figure 18: Part3: Q4 Source code in the editor

```
maarif@mujaabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 0s
^ gcc Q4.c -pthread -o Q4

maarif@mujaabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 0s
^
```

Figure 19: Figure 14: Q4 Successful compilation (gcc -pthread -o q4 Q4.c)

```
maarif@mujaabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 0s
^ ./Q4
Reporting thread: Waiting for departments to finish...
Department 2: Starting marks processing...
Department 1: Starting marks processing...
Department 4: Starting marks processing...
Department 3: Starting marks processing...
Department 1: Finished processing. Departments ready: 1/4
Department 2: Finished processing. Departments ready: 2/4
Department 3: Finished processing. Departments ready: 3/4
Department 4: Finished processing. Departments ready: 4/4

=== FINAL CLASS REPORT ===
All departments have submitted their marks.
Generating final class report...
Report completed successfully.

maarif@mujaabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 5s
^
```

Figure 20: Figure 15: Q4 Program execution showing department processing progress

Reflection

I developed a multithreaded program where four worker threads represent academic departments processing marks independently, and a reporting thread waits for all departments to finish before generating the final class report. Synchronization uses a mutex to protect the shared 'departments_ready' counter and a condition variable for the reporting thread to wait efficiently.

Each department thread simulates processing (with sleep), increments the counter under mutex protection, and signals the condition variable when the last department finishes. The reporting thread locks the mutex, waits on the condition variable until the counter reaches four, then proceeds to print the final report.

Condition variables are preferred over busy waiting because they allow the reporting thread to relinquish the CPU while waiting, rather than consuming cycles in a loop checking the condition. This is more efficient and responsive, especially when the wait time is variable or indefinite. In this scenario, departments finish at different times (simulated by varying sleep durations), so busy waiting would waste resources polling a condition that changes infrequently. Condition variables provide a clean mechanism for threads to sleep until a specific state change occurs, improving system utilization and responsiveness.

Question Five (20 Marks)

Multithreaded Inventory Management System (Mini Project)

Source Code

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

#define NUM_CATEGORIES 4

// Inventory structure
typedef struct {
    const char* name;
    int quantity;
} InventoryItem;

// Shared inventory
InventoryItem inventory[NUM_CATEGORIES];

// Mutex for protecting inventory
pthread_mutex_t inventory_mutex;

// Total updates counter
int total_updates = 0;

// Categories
```

```

const char* category_names[] = {
    "Electronics",
    "Groceries",
    "Clothing",
    "Stationery"
};

void* update_inventory(void* arg) {
    int category_idx = *(int*)arg;
    const char* category = category_names[category_idx];

    // Each thread performs 5 updates
    for (int i = 0; i < 5; i++) {
        // Simulate processing delay
        sleep(1);

        // Lock mutex before accessing shared inventory
        pthread_mutex_lock(&inventory_mutex);

        // Update inventory: random restock or sale
        int change = (rand() % 3) - 1; // -1, 0, or 1
        inventory[category_idx].quantity += change;
        if (inventory[category_idx].quantity < 0) {
            inventory[category_idx].quantity = 0; // Don't go negative
        }

        total_updates++;

        printf("[%s] Update #%d: Changed by %d. New stock level: %d\n",
            category, i+1, change, inventory[category_idx].quantity);

        pthread_mutex_unlock(&inventory_mutex);
    }

    return NULL;
}

int main() {
    pthread_t threads[NUM_CATEGORIES];
    int category_indices[NUM_CATEGORIES];

    // Initialize inventory
    for (int i = 0; i < NUM_CATEGORIES; i++) {
        inventory[i].name = category_names[i];
        inventory[i].quantity = 50; // Initial stock
        category_indices[i] = i;
    }

    // Initialize mutex
    pthread_mutex_init(&inventory_mutex, NULL);

    // Seed random number generator
    srand(time(NULL));

    printf("=== Supermarket Inventory Management System ===\n");
    printf("Initial inventory:\n");
    for (int i = 0; i < NUM_CATEGORIES; i++) {
        printf("  %s: %d units\n", inventory[i].name, inventory[i].quantity);
    }
    printf("\nStarting inventory updates...\n\n");

    // Create worker threads
    for (int i = 0; i < NUM_CATEGORIES; i++) {
        pthread_create(&threads[i], NULL, update_inventory, &category_indices[i]);
    }

    // Wait for all threads to complete
    for (int i = 0; i < NUM_CATEGORIES; i++) {
        pthread_join(threads[i], NULL);
    }

    // Destroy mutex
    pthread_mutex_destroy(&inventory_mutex);

    // Final inventory summary
    printf("\n=== FINAL INVENTORY SUMMARY ===\n");
    for (int i = 0; i < NUM_CATEGORIES; i++) {
        printf("  %s: %d units\n", inventory[i].name, inventory[i].quantity);
    }
    printf("\nTotal number of stock updates performed: %d\n", total_updates);
}

```

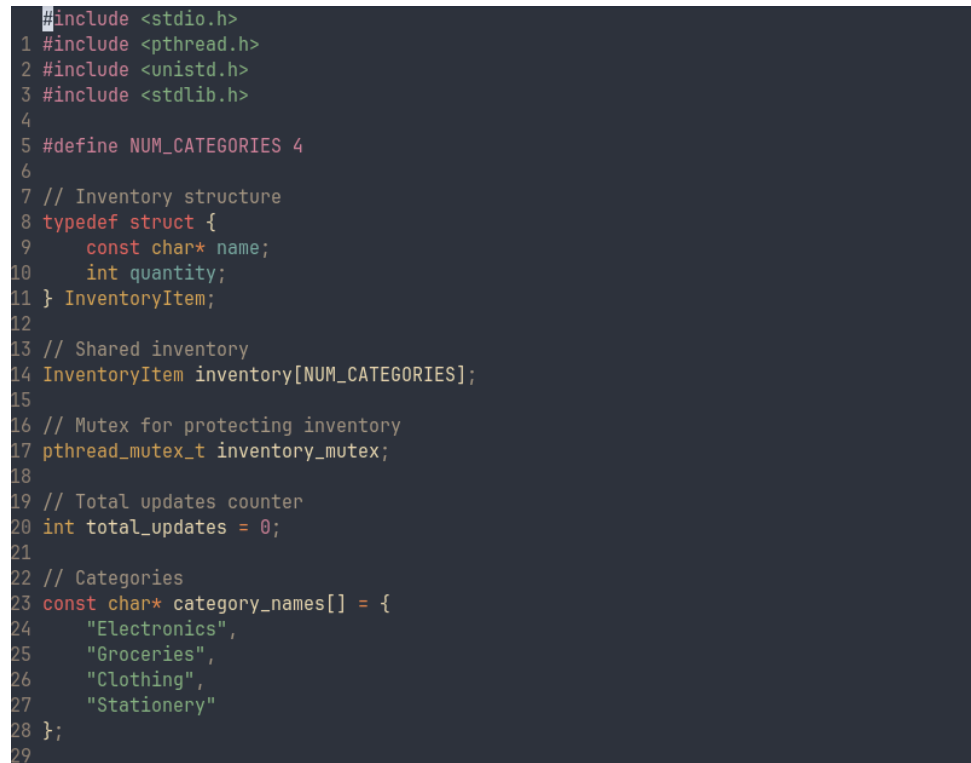


```

    printf("Inventory synchronization completed successfully.\n");
    return 0;
}

```

Screenshots

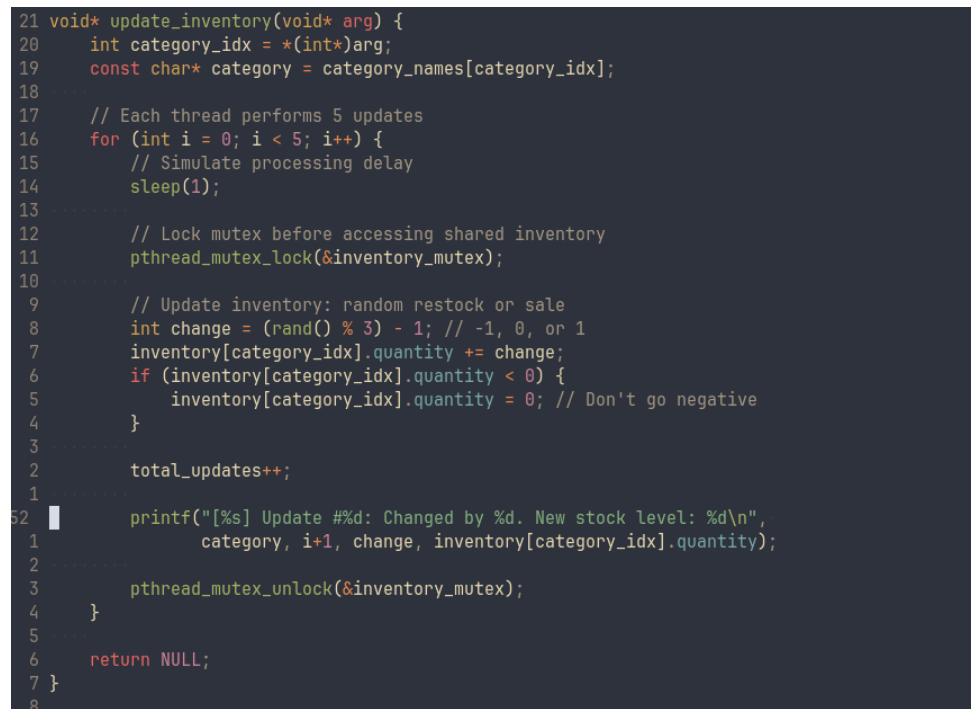


```

#include <stdio.h>
1 #include <pthread.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 #define NUM_CATEGORIES 4
6
7 // Inventory structure
8 typedef struct {
9     const char* name;
10    int quantity;
11 } InventoryItem;
12
13 // Shared inventory
14 InventoryItem inventory[NUM_CATEGORIES];
15
16 // Mutex for protecting inventory
17 pthread_mutex_t inventory_mutex;
18
19 // Total updates counter
20 int total_updates = 0;
21
22 // Categories
23 const char* category_names[] = {
24     "Electronics",
25     "Groceries",
26     "Clothing",
27     "Stationery"
28 };
29

```

Figure 21: Figure 17: Q5 Source code in the editor



```

21 void* update_inventory(void* arg) {
20     int category_idx = *(int*)arg;
19     const char* category = category_names[category_idx];
18
17     // Each thread performs 5 updates
16     for (int i = 0; i < 5; i++) {
15         // Simulate processing delay
14         sleep(1);
13
12         // Lock mutex before accessing shared inventory
11         pthread_mutex_lock(&inventory_mutex);
10
9         // Update inventory: random restock or sale
8         int change = (rand() % 3) - 1; // -1, 0, or 1
7         inventory[category_idx].quantity += change;
6         if (inventory[category_idx].quantity < 0) {
5             inventory[category_idx].quantity = 0; // Don't go negative
4         }
3
2         total_updates++;
1
52     printf("[%s] Update #d: Changed by %d. New stock level: %d\n",
1         category, i+1, change, inventory[category_idx].quantity);
2
3         pthread_mutex_unlock(&inventory_mutex);
4     }
5
6     return NULL;
7 }
8

```

Figure 22: Part2 : Q5 Source code in the editor

```

22 int main() {
21     pthread_t threads[NUM_CATEGORIES];
20     int category_indices[NUM_CATEGORIES];
19     for (int i = 0; i < NUM_CATEGORIES; i++) {
18         inventory[i].name = category_names[i];
17         inventory[i].quantity = 50; // Initial stock
16         category_indices[i] = i;
15     }
14     pthread_mutex_init(&inventory_mutex, NULL);
13     srand(time(NULL));
12
11     printf("=== Supermarket Inventory Management System ===\n");
10     printf("Initial inventory:\n");
9     for (int i = 0; i < NUM_CATEGORIES; i++) {
8         printf(" %s: %d units\n", inventory[i].name, inventory[i].quantity);
7     }
6     printf("\nStarting inventory updates...\n\n");
5
4     for (int i = 0; i < NUM_CATEGORIES; i++) {
3         pthread_create(&threads[i], NULL, update_inventory, &category_indices[i]);
2     }
1     for (int i = 0; i < NUM_CATEGORIES; i++) {
83         pthread_join(threads[i], NULL);
1     }
2     pthread_mutex_destroy(&inventory_mutex);
3     printf("\n=== FINAL INVENTORY SUMMARY ===\n");
4     for (int i = 0; i < NUM_CATEGORIES; i++) {
5         printf(" %s: %d units\n", inventory[i].name, inventory[i].quantity);
6     }
7     printf("\nTotal number of stock updates performed: %d\n", total_updates);
8     printf("Inventory synchronization completed successfully.\n");
9
10     return 0;
11 }

```

Figure 23: Part3 : Q5 Source code in the editor

```

maarif@nujabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 0s
^ gcc Q5.c -pthread -o Q5

maarif@nujabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 0s
^

```

Figure 24: Figure 18: Q5 Successful compilation (gcc -pthread -o q5 Q5.c)

```

maarif@nujabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 0s
^ ./Q5
=== Supermarket Inventory Management System ===
Initial inventory:
Electronics: 50 units
Groceries: 50 units
Clothing: 50 units
Stationery: 50 units

Starting inventory updates...

[Groceries] Update #1: Changed by 1. New stock level: 51
[Stationery] Update #1: Changed by -1. New stock level: 49
[Electronics] Update #1: Changed by 1. New stock level: 51
[Clothing] Update #1: Changed by 1. New stock level: 51
[Groceries] Update #2: Changed by 0. New stock level: 51
[Stationery] Update #2: Changed by 1. New stock level: 50
[Electronics] Update #2: Changed by 1. New stock level: 52
[Clothing] Update #2: Changed by 1. New stock level: 52
[Electronics] Update #3: Changed by 1. New stock level: 53
[Groceries] Update #3: Changed by 1. New stock level: 52
[Stationery] Update #3: Changed by 1. New stock level: 51
[Clothing] Update #3: Changed by 0. New stock level: 52
[Groceries] Update #4: Changed by -1. New stock level: 51
[Stationery] Update #4: Changed by -1. New stock level: 50
[Electronics] Update #4: Changed by -1. New stock level: 52
[Clothing] Update #4: Changed by -1. New stock level: 51
[Groceries] Update #5: Changed by 0. New stock level: 51
[Stationery] Update #5: Changed by -1. New stock level: 49
[Electronics] Update #5: Changed by 1. New stock level: 53
[Clothing] Update #5: Changed by 0. New stock level: 51

=== FINAL INVENTORY SUMMARY ===
Electronics: 53 units
Groceries: 51 units
Clothing: 51 units
Stationery: 49 units

Total number of stock updates performed: 20
Inventory synchronization completed successfully.

maarif@nujabi in repo: SystemsProgramming/TermPaper on P main [!?] via C v16.1.1-gcc via t v0.15.1 took 5s
^

```

Figure 25: Figure 19: Q5 Thread execution showing category stock updates

Reflection

I developed a multithreaded inventory management system for a supermarket with four categories: Electronics, Groceries, Clothing, and Stationery. Each category is processed by a dedicated worker thread that updates inventory quantities (simulating restocking by adding a random amount), displays the category and updated stock level, and uses `sleep()` to simulate processing delays.

To prevent race conditions on shared inventory data, I used a POSIX mutex to protect critical sections where inventory quantities and the total update counter are modified. Each thread locks the mutex before accessing shared data, performs its update, then unlocks it, ensuring exclusive access during modifications.

After all worker threads complete (via `pthread_join()`), the main thread displays: a final inventory summary showing stock levels for all categories, the total number of stock updates performed (incremented atomically under the mutex), and a confirmation of successful synchronization.

Challenges included ensuring the mutex was properly locked/unlocked in all code paths (including early returns) and avoiding deadlocks. I solved this by consistently using lock/unlock pairs and keeping critical sections minimal. Lessons learned: Mutexes are essential for protecting shared mutable state in concurrent systems, but must be used judiciously to avoid performance bottlenecks. Combining thread-specific work (like category processing) with synchronized shared-state updates is a common and effective pattern in inventory systems.

Problems Encountered in the Study

During the implementation of this term paper, I encountered several challenges that are common in multithreaded programming. Firstly, debugging race conditions proved difficult because they are non-deterministic and often depend on thread timing. For Question Two, observing the inconsistent balances required multiple runs to notice the pattern of lost updates. Secondly, managing thread lifetimes and ensuring proper synchronization to avoid deadlocks was tricky, especially in Question Five where multiple threads access shared inventory. Thirdly, correctly using condition variables in Question Four required careful attention to the while loop in `pthread_cond_wait` to handle spurious wakeups. Fourthly, compiling with the correct flags (not forgetting `-pthread`) caused initial linkage errors. Finally, ensuring that arguments passed to threads remain valid (avoiding dangling pointers) required thoughtful design, such as using arrays with sufficient scope. These experiences reinforced the importance of systematic testing, careful synchronization design, and understanding the underlying POSIX thread semantics.

References

- POSIX Threads Programming, Lawrence Livermore National Laboratory Computing, <https://computing.llnl.gov/tutorials/pthreads/>
- GNU C Library Manual, Free Software Foundation, <https://www.gnu.org/software/libc/manual/>
- Lecture notes and textbook for CCS 3105: Systems Programming, Dedan Kimathi University of Technology
- Online tutorials and reference materials used during implementation